ROOTCAUSE SERVER

12 · Pruebas y calidad

Documento: 12-testing-and-quality.md **Versión analizada:** 0.2.0 **Commit:** 767a67b **Fecha del análisis:** 2026-08-26

CONTENIDO

- Tipos de prueba existentes
- Distribución por binario de prueba
- Cómo ejecutarlas
- Datos y fixtures
- Qué cubre cada área
  - rootcause-core — el dominio (92 pruebas)
  - rootcause-agent — el sensor (59 pruebas)
  - rootcause-server — el plano de control (77 + 25 pruebas)
  - Integración (25 pruebas)
- Cobertura
- Herramientas de análisis estático
- Integración continua
- Criterios de aceptación
- Casos límite ya cubiertos
- Módulos y áreas sin prueba directa
- Propuesta priorizada de pruebas faltantes

> Qué se prueba, cómo se ejecuta, qué cubre CI, qué queda sin cubrir y una propuesta priorizada de pruebas faltantes.
>
> Las cifras de este documento provienen de una ejecución real de `cargo test --workspace --all-targets --locked` sobre el commit `2efedb2`, el 2026-08-26. Resultado: **253 pruebas, 0 fallos**.

Tipos de prueba existentes

Tipo	Dónde	Cantidad	Qué verifica
Unitarias de dominio	Módulos `#[cfg(test)]` de `rootcause-core`	92	Clasificación, política, reglas, postura, runbooks
Unitarias del agente	Módulos `#[cfg(test)]` de `rootcause-agent`	59	Parsers por plataforma, lista blanca, huellas, transporte
Unitarias del servidor	Módulos `#[cfg(test)]` de `rootcause-server` (biblioteca)	75	Configuración, perímetro, autenticación, cabeceras, errores, almacenamiento, vigilante
Unitarias del binario	`main.rs` del servidor	2	Catálogo no vacío; política imprimible y recargable
Integración	`tests/control_plane.rs`	25	El router completo en proceso, extremo a extremo
Humo real	`scripts/smoke.sh`	1 guion	Binarios reales, socket real, máquina real

Tipo	Dónde	Cantidad	Qué verifica
Guardianes	3 scripts Python	3	Invariantes que un revisor humano olvidaría

Total: 253 pruebas automatizadas más un guion de humo y tres guardianes.

Distribución por binario de prueba

```
rootcause_agent  (unittests src/main.rs)      59 pruebas
rootcause_core   (unittests src/lib.rs)       92 pruebas
rootcause_server (unittests src/lib.rs)       75 pruebas
rootcause_server (unittests src/main.rs)       2 pruebas
control_plane    (tests/control_plane.rs)     25 pruebas
```

Cómo ejecutarlas

Suite completa:

```
cargo test --workspace --all-targets --locked
```

Solo el dominio:

```
cargo test -p rootcause-core
```

Solo la integración del plano de control:

```
cargo test -p rootcause-server --test control_plane
```

Pruebas de documentación:

```
cargo test --workspace --doc --locked
```

Humo real, con binarios y socket:

```
bash scripts/smoke.sh
```

Los tres guardianes:

```
python3 scripts/guard_console.py && python3 scripts/guard_claims.py && python3
scripts/guard_encoding.py
```

> **Nota de entorno.** En la máquina Windows de este análisis, `cargo` se ejecuta dentro de WSL porque el toolchain de Windows no tiene un enlazador utilizable. En CI las tres plataformas se compilan y prueban de forma nativa.

Datos y fixtures

No hay archivos de fixture. Todos los datos de prueba están en el código, lo que evita que un fixture y su prueba se desincronicen.

Fuente de datos	Ubicación	Uso
`detect::fixtures`	`detect/mod.rs`, solo en compilación de test	`agent_id()`, `registration(role)`, `sample()`, `sample_at()` compartidos por los cinco detectores
Salidas de herramientas	Literales dentro de cada prueba de `net.rs`, `authlog.rs`, `baseline.rs`	Texto real de `ss`, `netstat`, `journalctl`, `ufw`, `nft`, `netsh`…
Ayudantes de integración	`tests/control_plane.rs`	`app()`, `registration()`, `sample()`, `authorized()`, `ingest()`
Base de datos	`sqlite::memory:`	Cada prueba arranca con una base limpia

El identificador de agente de las fixtures es `5eed0000-0000-0000-0000-000000000001`: una semilla («seed») reconocible a simple vista en cualquier volcado.

Qué cubre cada área

`rootcause-core` — el dominio (92 pruebas)

Módulo	Ejemplos de lo que fija
`security`	Loopback no es exposición; comodines son públicos; RFC1918 y ULA son privados; **lo ilegible es público**; el catálogo está ordenado y sin duplicados; IPv6 va entre corchetes; se detecta el bit de escritura para otros
`policy`	La política por omisión es válida; se rechazan umbrales invertidos, fuera de rango y `cpu_sustained_samples = 0`; loopback nunca produce riesgo; una base de datos pública es crítica y una privada baja un escalón; el borde puede publicar HTTP; la política va y vuelve por JSON; **un campo desconocido se rechaza**
`detect`	Los identificadores son únicos y llevan el prefijo de su categoría; toda regla declara pregunta y técnica; un servidor sano no produce hallazgos; una política inválida se rechaza al construir el motor; el conteo coincide con el catálogo
`intrusion`	Umbrales de ráfaga, barrido, campaña distribuida y acceso tras ráfaga
`resource`	Un pico aislado no es saturación; la racha sí
`integrity`	El cambio solo se detecta con línea base previa
`posture`	Una flota limpia saca 100; los resueltos no cuentan; el techo funciona
`runbook`	**Ningún comando es destructivo**, y la contención va precedida de inspección
`lib`	El protocolo actual está entre los admitidos

`rootcause-agent` — el sensor (59 pruebas)

Módulo	Cobertura
`net`	Los cuatro parsers contra salidas reales; agregación de pares; corchetes IPv6; comodines; estados en inglés y en español
`authlog`	`sshd` en Linux y eventos 4624/4625 en Windows; agregación; recorte
`baseline`	Los cinco parsers de firewall y los dos de actualizaciones
`integrity`	Huella conocida; lista de vigilancia; rutas inexistentes frente a ilegibles

Módulo	Cobertura
`probe`	La lista blanca acepta rutas absolutas y `.exe`; **rechaza lo que no está en ella**
`main`	`validate_transport`, `parse_label`, `endpoint`, `stable_agent_id`

`rootcause-server` — el plano de control (77 + 25 pruebas)

Módulo	Cobertura
`config`	Configuración sana aceptada; token corto o con espacios rechazado; modo inseguro solo en loopback; URL no SQLite rechazada; `--trust-forwarded-for` sin proxy rechazado; política inválida detiene el arranque; **el contrato de la CLI se valida con `debug_assert`**
`defense`	Cubeta, bloqueo, expiración, desalojo, poda
`auth`	Dirección del cliente con y sin proxy; cabecera malformada; solo `Bearer`; comparación de token contra prefijos y sufijos
`headers`	La CSP no admite `unsafe-inline`, `unsafe-eval`, `http:` ni `*`; toda cabecera declarada es válida; HSTS con su duración
`error`	Cada error a su estado; el cuerpo del bloqueo nombra su código; `Retry-After` nunca es cero; **un error interno no filtra la causa**
`storage`	Alta y actualización de activos, incidentes, líneas base, presión, defensa, auditoría y retención; **que un agregado lea toda la tabla y no la primera página**, que el resumen puntúe igual que los incidentes que resume, y que el rastro de auditoría se pueda leer entero o paginado
`watchdog`	Se levanta el silencio y se cierra al volver
`ui`	Los cuatro activos están embebidos; las rutas del API no caen a la consola; una ruta desconocida sirve el cascarón
`api`	`build_exposure`, `build_topology`, `render_export`, `render_metrics`
`state`	El informe de endurecimiento refleja la configuración; un evento de defensa se persiste y se cuenta

Integración (25 pruebas)

Nombres que describen el comportamiento verificado, no la implementación:

- `health_is_public_and_the_api_is_not`
- `a_wrong_token_is_rejected_and_the_body_says_why`
- `repeated_wrong_tokens_lock_the_caller_out`
- `every_response_carries_the_security_headers`
- `the_console_is_served_and_is_not_cached_as_evidence`
- `a_healthy_server_produces_no_incidents`
- `an_exposed_database_becomes_a_critical_incident_with_a_runbook`
- `a_closed_port_resolves_its_own_incident_on_the_next_cycle`
- **`a_finding_stays_open_when_its_surface_could_not_be_inspected`**
- `a_brute_force_burst_reaches_the_threat_report`
- `a_changed_watched_file_is_detected_only_after_a_baseline_exists`
- `the_status_endpoint_reports_posture_and_hardening`
- `the_exposure_report_lists_the_service_behind_the_port`
- `the_topology_places_an_exposed_host_behind_the_internet_node`
- `an_unsupported_protocol_version_is_refused`
- `telemetry_for_an_unknown_asset_is_refused`

- `an_agent_without_a_security_surface_is_told_so`
- `a_collection_gap_is_reported_back_to_the_agent`
- `an_incident_can_be_acknowledged_and_the_change_is_audited`
- `the_rule_catalog_is_published_with_its_attack_mapping`
- `an_invalid_filter_value_is_rejected_instead_of_ignored`
- `the_policy_in_force_can_be_read_back`
- `metrics_are_exposed_in_prometheus_format`
- `the_evidence_export_is_newline_delimited_json`
- `an_asset_detail_carries_its_own_posture_and_history`

La tercera empezando por el final de la lista destacada, `a_finding_stays_open_when_its_surface_could_not_be_inspected`, es la prueba de REQ-SEC-003: verifica que un hallazgo **no** se cierra cuando la superficie que lo produjo no se pudo inspeccionar. Corresponde al arreglo del commit `790036f`.

Cobertura

Aspecto	Valor
Herramienta	`cargo-llvm-cov` 0.9.0
Alcance del comando	`--workspace`
Umbral que hace fallar CI	**80 % de líneas** (`--fail-under-lines 80`)
Artefacto	`lcov.info`, subido siempre (`if: always()`)
Cobertura medida el 2026-08-26	**89,34 % de líneas** · 88,05 % de regiones · 88,15 % de funciones

Es decir, el umbral de CI se supera con **más de nueve puntos de margen**.

Cobertura por archivo, de mayor a menor —los que quedaron fuera del recorte de la salida no aparecen aquí, pero sí cuentan en el total—:

Archivo	Líneas cubiertas
`server/error.rs`	100,00 %
`core/lib.rs`	100,00 %
`server/auth.rs`	99,04 %
`core/detect/resource.rs`	98,65 %
`server/defense.rs`	98,55 %
`core/posture.rs`	98,42 %
`server/config.rs`	98,32 %
`server/storage.rs`	98,31 %
`server/ui.rs`	96,08 %
`core/runbook.rs`	95,22 %
`core/policy.rs`	92,73 %
`server/api.rs`	90,64 %

Archivo	Líneas cubiertas
`server/state.rs`	90,32 %
`core/security.rs`	90,18 %
`server/headers.rs`	90,00 %
`server/watchdog.rs`	88,65 %
`core/models.rs`	86,56 %
`server/main.rs`	**6,67 %**

El único valor bajo es el esperado. `main.rs` abre sockets, instala suscriptores globales y espera señales del sistema: nada de eso se puede ejercitar desde una prueba unitaria, y es exactamente lo que cubre `scripts/smoke.sh` arrancando el binario de verdad. La distribución confirma lo que dice la tabla de más abajo sobre módulos sin prueba directa.

> **Observación de coherencia.** El paso de CI se llama «Medir cobertura del núcleo de detección», pero el comando cubre **todo el workspace**. El nombre se quedó desactualizado; el comando es el correcto. Recogido como hallazgo menor en 15 · Riesgos.

Para reproducir la medición:

```
cargo llvm-cov --workspace --locked --summary-only
```

Herramientas de análisis estático

Herramienta	Alcance	Configuración	Severidad
`rustfmt`	Todo el Rust	`rustfmt.toml`: ancho 100, 4 espacios, Unix	Falla CI
`clippy`	Workspace, todos los objetivos, **tres sistemas**	`-D warnings` + lints del workspace	Falla CI
`rustdoc`	API pública	`RUSTDOCFLAGS: -D warnings`	Falla CI
`cargo deny` 0.20.2	Dependencias	`deny.toml`	Falla CI. **Ejecutado el 2026-08-26: `advisories ok, bans ok, licenses ok, sources ok`**
`cargo audit` 0.22.2	Avisos publicados	`.cargo/audit.toml`	Falla CI. **Ejecutado el 2026-08-26: 315 dependencias analizadas contra 1 226 avisos, sin hallazgos**
`actionlint` 1.7.12	Workflows	—	Falla CI
`zizmor` 1.5.2	Workflows, persona `regular`	—	Falla CI
`markdownlint-cli2` 0.18.1	Todo el Markdown salvo `target`	`.markdownlint-cli2.jsonc`	Falla CI
`hadolint` 2.14.0	`Dockerfile`	`.hadolint.yaml`	Falla CI
`trivy`	Imagen construida	`CRITICAL` con corrección disponible	Falla CI

Herramienta	Alcance	Configuración	Severidad
`CodeQL`	`javascript-typescript`, `actions`	Workflow propio	Informa
`gitleaks`	Histórico completo	`.gitleaks.toml`	Falla el workflow de seguridad

Lints del workspace declarados en `Cargo.toml`:

```
[workspace.lints.rust]
unsafe_code = "forbid"
missing_debug_implementations = "warn"
unused_qualifications = "warn"

[workspace.lints.clippy]
todo = "deny"
unimplemented = "deny"
dbg_macro = "deny"
```

`todo!()`, `unimplemented!()` y `dbg!()` **no compilan**. Es coherente con un producto de seguridad: un `todo!()` en producción es un pánico esperando a ocurrir.

Integración continua

15 trabajos en `ci.yml`:

#	Trabajo	Ejecuciones
1	Formato	1
2	Clippy	3 (Linux, Windows, macOS)
3	Pruebas	**6** (3 sistemas × 2 toolchains: `1.95.0` y `stable`)
4	Documentación del API	1
5	Cobertura	1
6	Dependencias y licencias	1
7	Versión mínima de Rust	1
8	Consola embebida	1
9	Coherencia de lo que se afirma	1
10	Workflows	1
11	Markdown	1
12	Integridad del texto	1
13	Imagen de contenedor	1
14	Prueba de humo extremo a extremo	1
15	**Todo en verde** (puerta final)	1

La puerta final es lo más interesante del diseño. Depende de los catorce trabajos anteriores, corre con `if: always()` y recorre `join(needs.*.result, ' ')` haciendo fallar la rama si algún resultado no es `success`.

Eso incluye `skipped` y `cancelled`: **un control que no llegó a ejecutarse no cuenta como aprobado**.

Trabajos adicionales en `security.yml` (avisos, secretos en el histórico, dependencias, workflows, SBOM), ejecutados también cada lunes a las 06:17 UTC.

Criterios de aceptación

`AGENTS.md` define la puerta de finalización de todo cambio:

```
cargo fmt --all -- --check && cargo clippy --workspace --all-targets -- -D warnings && cargo test --workspace && bash scripts/smoke.sh && python3 scripts/guard_console.py && python3 scripts/guard_claims.py
```

Y seis invariantes que CI hace cumplir:

1. Ningún comando de runbook es destructivo, y la contención va precedida de inspección.
2. La CSP no admite `unsafe-inline` ni `unsafe-eval`; la consola no tiene manejadores en línea, origen externo ni sumideros de marcado.
3. El número de reglas de la documentación es el que compila.
4. Toda acción de GitHub está pinneada a SHA completo y todo workflow declara sus permisos.
5. Una política de detección contradictoria impide arrancar.
6. Ningún control de CI puede quedar sin ejecutar.

Casos límite ya cubiertos

Caso	Prueba
Dirección de enlace ilegible	`unparseable_addresses_fail_towards_the_worse_case`
Métrica no finita o fuera de rango	`metric_validation_rejects_impossible_values`
Token con prefijo o sufijo del correcto	`token_comparison_rejects_prefixes_and_accepts_the_exact_secret`
Cabecera `X-Forwarded-For` malformada	`a_malformed_forwarded_header_falls_back_to_the_peer`
Conexión sin información de par	`a_connection_without_peer_information_gets_a_stable_placeholder`
`Retry-After` calculado como cero	`a_zero_retry_hint_is_never_sent`
Error interno con secreto en el mensaje	`an_internal_error_never_leaks_the_cause`
Primer ciclo de un archivo vigilado	`a_changed_watched_file_is_detected_only_after_a_baseline_exists`
Superficie no inspeccionada	`a_finding_stays_open_when_its_surface_could_not_be_inspected`
Campo desconocido en la política	`unknown_policy_fields_are_rejected`
Filtro con valor inválido	`an_invalid_filter_value_is_rejected_instead_of_ignored`
Contador de red que retrocede	Cubierto por `checked_sub` en `egress_rate`

Módulos y áreas sin prueba directa

Área	Estado	Por qué
`main.rs` del servidor: `run`, `announce`, `shutdown_signal`, `init_tracing`	Sin prueba unitaria	Abren sockets y suscriptores globales. **Cubiertos por `smoke.sh`**

Área	Estado	Por qué
`main.rs` del agente: bucle principal, `send_once`, `dry_run`	Sin prueba unitaria	Requieren red. **Cubiertos por `smoke.sh`**
`collector.rs`: `sample()`, `security()`	Sin prueba unitaria	Dependen del sistema operativo real
Ramas por plataforma de `net`, `authlog`, `baseline`	Los **parsers** sí; el despacho `cfg!(target_os)` no	El despacho se ejercita en la matriz de CI de tres sistemas
`console/app.js`	**Sin pruebas**	No hay ejecutor de JavaScript en el proyecto. Solo se comprueba sintaxis y `guard_console.py`
`console/styles.css`, `landing/`	Sin pruebas	Presentación
Los tres scripts `guard_*.py` y `sbom.py`	**Sin pruebas propias**	Se ejecutan en CI; un fallo suyo se nota al instante
Migraciones descendentes	No existen	No hay reversión
Comportamiento bajo concurrencia real	No probado	*Requiere validación* con carga

Propuesta priorizada de pruebas faltantes

#	Prioridad	Prueba propuesta	Qué protegería	Coste
1	**Alta**	Prueba de propiedad (fuzz ligero) sobre los cuatro parsers de `net.rs` y los dos de `authlog.rs` con entrada arbitraria	Que una salida inesperada de `ss` o `netstat` no produzca pánico ni datos absurdos. Es el código que interpreta texto de terceros	Medio
2	**Alta**	Ingesta concurrente: N sobres del mismo activo a la vez	Que la deduplicación por huella y el contador de ocurrencias resistan la concurrencia real de SQLite	Medio
3	**Alta**	Ingesta interrumpida a mitad, verificando el estado de la línea base	La no atomicidad identificada en 07	Alto
4	**Media**	Actualización de esquema: crear una base con `0001`, insertar filas, aplicar `0002` y comprobar que todo sigue legible	La compatibilidad que `0002` promete en su cabecera	Bajo
5	**Media**	Sobre de un agente `0.1` (sin `security`) contra un servidor `0.2`	La compatibilidad de protocolo declarada	Bajo
6	**Media**	Pruebas de la consola con un DOM sintético (por ejemplo `jsdom` o Deno) sobre `renderIncidents` y `renderExposure`	Las 1 177 líneas de `app.js` hoy solo tienen comprobación de sintaxis	Alto
7	**Media**	Verificación de que un nombre de equipo con `<script>` se muestra literal en la consola	La promesa de la construcción del DOM, hoy garantizada solo por revisión estática	Medio
8	**Baja**	Pruebas de los guardianes: alimentarlos con un archivo que debería rechazar	Que un guardián no pase a aprobarlo todo por una expresión regular rota	Bajo
9	**Baja**	Rendimiento de la ingesta con 100 activos y 12 muestras de historial	Detectar la degradación de `list_incidents` sin filtro, que hoy se llama en varias rutas	Medio
10	**Baja**	Verificación del archivo de política de ejemplo, si en el futuro se añade uno	Que el ejemplo publicado sea válido	Bajo

La n.º 1 es la más valiosa. Todo lo demás son estructuras propias y bien tipadas; los parsers son el único punto donde entra texto escrito por software de otros, y su comportamiento frente a formatos no previstos es la principal incógnita técnica de este repositorio.

Anterior: 11 · Seguridad · **Siguiente:** 13 · Despliegue y operación · **Índice:** README
